

Auto-Remediation from Logs — Client Scoping Note

To: VP, Platform Engineering | **Re:** Automating log-noise triage and incident remediation

The problem

Your on-call engineers read thousands of log lines a day to find the handful that matter. Most are harmless — health checks, cache warmups. A real minority are genuine incidents, and the same five or six keep recurring across services because nobody's stitching duplicates together. We're proposing an agent that reads every line, discards the noise for free, recognizes when it's seeing a repeat incident, and either tells on-call exactly what's wrong and how to fix it — or says it isn't sure and hands the case to a human, instead of guessing.

What "production-ready" means here

A demo version calls an LLM on every log line. That looks fine at five lines and fails at real volume three ways: it's **slow** (every line, including the 95% that are noise, waits on the model); it's **expensive** (the same handful of incidents get re-diagnosed hundreds of times a day); and it's **unsafe** (nothing stops the model inventing a "fix" outside your runbook, or guessing instead of admitting it isn't sure). Production-ready means noise never reaches the model, repeats are classified once, every remediation comes from a fixed approved list, and low-confidence cases are flagged for a human with reasoning attached — not silently guessed.

The trade-off we're making

We're optimizing for **cost and accuracy**, spending latency only where it earns its keep. Filtering and deduplicating before the model is ever called means the bulk of volume costs nothing and takes no time — but the handful of genuinely ambiguous incidents that do reach the model still take several real seconds, because we won't trade a wrong or invented answer for a faster one. If raw per-event speed mattered more than correctness, we'd shape this differently (see risk 3) — for a triage tool feeding a human queue, cost discipline and correctness matter more than shaving seconds off a decision nobody's blocked on synchronously.

Three biggest risks

1. **The model's judgment doesn't always match your taxonomy.** On genuinely ambiguous cases, classification can vary call-to-call, or land consistently on a defensible-but-different answer than your team would pick. *De-risk:* measured directly against a labeled set before go-live; consistent disagreement is a signal to sharpen the agent's instructions, not something to quietly trust.
2. **The cost story depends on how repetitive your logs actually are.** If a service starts emitting genuinely novel errors constantly, the dedup win shrinks. *De-risk:* monitor the unique-signature ratio in production so a regression shows up as a metric, not a surprise bill.
3. **A wrong classification could drive a bad action.** If this ever auto-executes remediations (not just recommends them), an error stops being an inconvenience. *De-risk:* nothing here auto-executes; every output carries a confidence score and an audit trail, and anything uncertain routes to a human by design.

Two-week scope

Week one: noise filter and dedup live against real log volume; closed-set remediation contract validated against your actual runbooks; baseline-vs-optimized benchmark harness producing real numbers, not projections.

Week two: confidence gating tuned against a labeled sample from your environment; audit trail wired into existing on-call tooling; a scale test at 10x volume; a live walkthrough of the benchmark so your team can re-run it independently.